

基于大模型编码智能体的自进化 eBPF 网络入侵防御系统

硕士学位论文开题报告

研究生：李兆曦

指导教师：徐恪教授

清华大学计算机科学与技术系

2026 年 6 月



目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

TCP/IP 跨层歧义持续催生 off-path 协议攻击

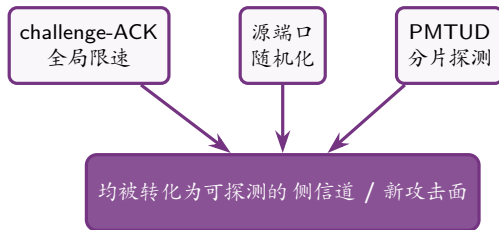
- TCP/IP 协议栈的跨层交互存在语义歧义，长期催生 **off-path**（链外）协议攻击——攻击者无需位于通信路径上，即可推断或注入连接状态：
 - challenge-ACK 侧信道¹（CVE-2016-5696，全局限速可被探测）
 - 混合 **IPID** 分配²、PMTUD 分片机制³、NAT / Wi-Fi 链路劫持⁴...
- 近期 **CCS'25** 工作⁵：利用 **PMTUD** 打破“IP 地址共享”场景下的 TCP 连接隔离
 - 单次攻击平均约 **220 s**，成功率约 **70%**
 - 实测 **50** 个真实网络中 **38** 个存在脆弱性

核心判断

协议攻击面并非偶发，而是植根于设计层面的跨层歧义，会被持续不断地重新发掘⁶。

规律：防御机制本身在制造新的攻击面

- 大量新攻击面恰由防御机制自身引入——“补丁”反而成为下一处侧信道：



启示

攻击与防御同源且相互转化 $\Rightarrow$ 防御须持续演化，一次性静态修补难以为继。

现有防御的三大结构性缺陷

1 零散

各攻击面分别打补丁，缺乏统一框架，防御策略碎片化、难以协同。

2 滞后

先有攻击曝光、再被动响应，从披露到修复存在显著时间窗口。

3 依赖人工编写内核代码

- 内核协议栈改动周期长：评审、回归、上线缓慢
- 易出错：一处缺陷即可导致网络中断或误伤正常流量
- 知识难以沉淀：经验留存于个人，复用困难

症结：防御的生产方式仍是人工、低速、难以累积的。

两项技术趋于成熟，使自动化防御成为可能

eBPF 内核可编程

- 热加载：无需修改内核源码、无需重启
- 贴近协议栈：XDP / TC 钩子在数据面线速处理
- 内置 **verifier**：对程序施加形式化安全验证（有界、内存安全、可终止）⁷

大模型编码智能体

- 能够编写、修改、调试系统级代码
- 具备工具调用、反思与迭代能力
- 已能在反馈闭环中自主推进复杂任务⁸⁻⁹
- 代表性产品 / 框架：Claude Code、Cursor、Codex、Devin、SWE-agent¹⁰⁻¹¹

契机

二者结合:eBPF 作可热插拔、可验证的内核执行底座, 智能体作自动化代码生产——使“由智能体持续维护内核防御”可行。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

核心研究问题

研究问题

能否让大模型编码智能体自主维护一套运行于内核的 **eBPF** 防御库，
使其持续进化，以应对不断涌现的协议攻击？

- 目标不是“一次性合成单个防御程序”，而是构建一个长期自主运转的防御系统；
- 核心在于自主性（探索—合成—验证—部署的闭环）与进化性（防御能力单调累积、不后退）。

关键挑战

1 可靠性

智能体生成的内核代码若存在缺陷, 可能导致网络中断或误伤正常流量——须做到“先验证安全、再上线, 异常可回滚”。

2 生成质量

智能体生成的 eBPF 须能编译通过、通过内核 **verifier**, 并有效拦截目标攻击而不误伤正常流量——这决定系统是否可用。

3 评测

如何验证其有效性: 对已知与未知攻击均能防御, 且性能开销可控。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

相关工作（一）：协议攻防与 eBPF 运行时防御

协议攻击与检测

- 攻击：off-path 协议攻击系列工作（challenge-ACK、IPID、PMTUD 等）⁵⁻⁶
- 检测：SCENT¹²、SCAD¹³ 等侧信道检测方法
- 现状：防御零散、滞后，缺少统一且持续演化的框架

eBPF 运行时防御

- 系统：Katran¹⁴、Cilium¹⁵、Falco¹⁶、Tetragon¹⁷——能力强，但策略均由人工编写
- 验证：PREVAIL⁷、K2¹⁸ 等围绕 verifier 的程序分析与优化

相关工作（二）：大模型用于安全 / 编码智能体

大模型 × 安全

- 进攻：PentestGPT¹⁹、Big Sleep²⁰
- 合成 **eBPF**：Kgent / KEN²¹——由自然语言合成 eBPF，是最直接的对比基线；但一次性合成、不持续维护

编码智能体（工程载体）

- Claude Code / Cursor / Codex / Devin / SWE-agent^{10-11,22}：已能自主编写、修改与调试代码，但属通用工具、不含内核 / **eBPF** 领域能力——本文为此设计专用 **harness**
- 迭代思路可借鉴 FunSearch、AlphaEvolve⁸⁻⁹（以测试反馈驱动模型反复修改代码）

定位对比：本文与代表性工作

对比对象	已有范式	本文范式
Kgent / KEN ²¹ Tetragon / Falco ¹⁶⁻¹⁷ 小模型（权重学习）	一次性合成 eBPF 人工编写策略 知识沉淀为权重	长期自主维护的防御库 自进化地生成与更新策略 知识沉淀为可验证代码

- 共同差异：由一次合成 / 人工 / 权重 → 持续、自主、可验证的代码演化。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

总体思路：由智能体持续维护内核 eBPF 防御

基本做法

将“发现攻击 生成/修改 **eBPF** 验证安全 加载 评估效果 迭代”组织为一个自动化闭环；防御能力以 **eBPF** 代码的形式持续积累于“防御库”中（本文称之为“防御演化”）。

为何以代码（而非模型权重）为载体

- 逐条增补与修正，可累积、不遗忘；每条防御对应一段代码，可读、可审计
- 运行于 XDP，线速执行；出错可回滚，并由 verifier 保证不致内核崩溃

主要研究内容 (两项)

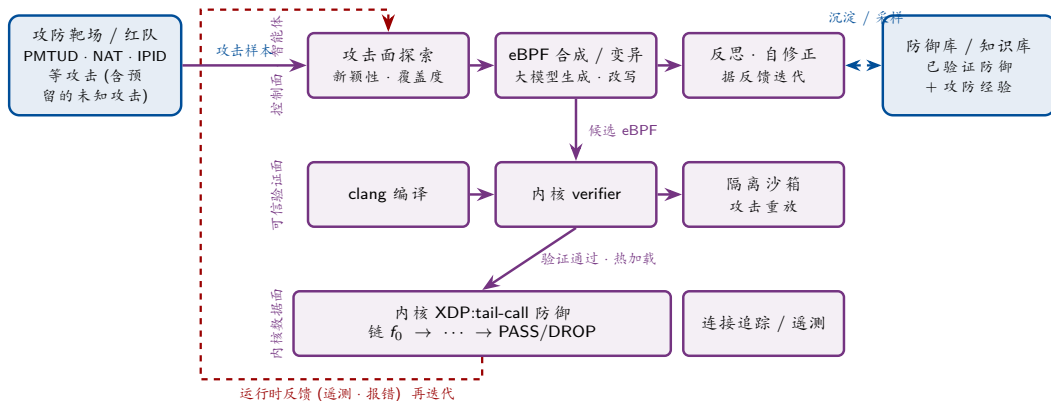
① 一套完整的专用 harness

将“由编码智能体生成 eBPF 防御并保证其可用、安全”的全过程, 封装为一套完整的领域专用 harness: 涵盖防御程序的生成、编译、内核 verifier 校验、沙箱测试、加载与运行时反馈; 并解决生成质量 (可编译、通过 verifier、不误伤正常流量) 与运行安全 (异常时自动回滚)。

② 面向真实协议攻击的防御与评测

以 PMTUD / NAT / IPID 等真实攻击构建攻防靶场, 由系统自动生成防御; 度量检测率、误报率与性能开销, 并与人工规则及现有工具对比。

系统分层架构



控制面 (智能体) 生成候选 可信验证面把关 内核数据面执行; 反馈回灌驱动持续迭代, 有效防御沉淀入知识库。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

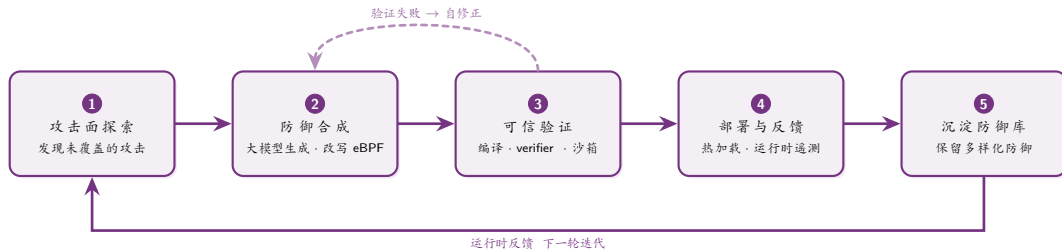
④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

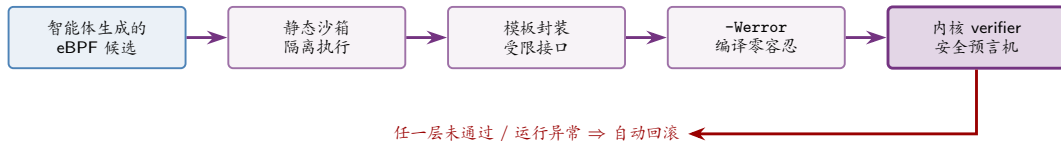
⑦ 预期成果

技术路线：防御演化闭环（五阶段）



每轮迭代将新发现的攻击面转化为一段经 verifier 背书的 eBPF 防御，并沉淀入防御库。

可信保障：四层安全约束 + 自动回滚



思路：现成安全设施 + 智能体闭环

这四层（沙箱、模板、-Werror、内核 verifier）都是现成的基础设施；本文不重造它们，而是把它们接入智能体闭环——以编译 / verifier 报错作反馈驱动自修正，沙箱重放把关，异常即自动回滚，从而可靠地实现内核代码自动化。

工程基线：已有开源系统积累

PacketScope / Guarder

- **XDP** 数据面：在网卡驱动层线速处理报文
- **bpf2go** 一键编译：Go 侧封装，eBPF 程序构建与加载流程化
- 规则热加载：策略可在运行时动态更新，无需重启
- 已具备 **Agent** 可编程 **eBPF** 原型，配套 **312** 项测试

已具备数据面、编译链与热加载基础设施，闭环可在现有底座上快速搭建。

评测方法

思路

以真实协议攻击构建攻防靶场, 由系统自动生成防御, 考察其检测率、误报率与性能开销; 并对未参与迭代的未知攻击单独评测。

度量指标

- 检测率 (是否成功拦截)
- 误报率 (误伤正常流量)
- 性能开销 (XDP 吞吐 / 时延)
- 自动化程度 (人工介入量)

对比基线

- 人工编写的规则 (现状)
- Kgent / KEN(一次性合成)
- Tetragon / Falco(人工策略)

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

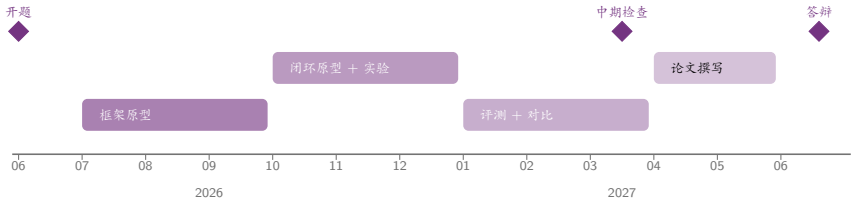
⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

工作计划（2026.06 – 2027.06）

时间	工作内容
2026.06	开题报告
2026.07 – 09	防御演化框架原型：形式化定义 + 系统骨架
2026.10 – 12	闭环原型 + 初步实验（探索 / 合成 / 验证 / 部署 / 档案）
2027.01 – 03	评测 + 对比实验 + 中期检查
2027.04 – 05	论文撰写、投稿 + 预答辩 + 送审
2027.06	学位论文答辩



目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 工作计划

⑦ 预期成果

预期成果

- 学位论文：完成硕士学位论文 1 篇。
- 开源系统：开源自进化 **eBPF** 防御系统、防御演化专用 **harness** 及配套攻防评测基准。
- 社区贡献：对发现的协议 / 实现缺陷，向 **IETF / Linux** 进行负责任披露。

成果定位

既贡献新范式与系统，也贡献可复现的评测基准，推动“智能体自主维护内核安全”方向的后续研究。

参考文献 I

- [1] CAO Y, WANG Z, QIAN Z, et al. Off-Path TCP Exploits: Global Rate Limit Considered Dangerous[C]//25th USENIX Security Symposium. 2016: 209-225.
- [2] FENG X, FU C, LI Q, et al. Off-Path TCP Exploits of the Mixed IPID Assignment[C]//Proc. 2020 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2020: 1323-1335. DOI: 10.1145/3372297.3417884.
- [3] FENG X, LI Q, SUN K, et al. PMTUD is not Panacea: Revisiting IP Fragmentation Attacks against TCP[C]//29th Network and Distributed System Security Symposium (NDSS). 2022. DOI: 10.14722/ndss.2022.24381.
- [4] YANG Y, FENG X, LI Q, et al. Exploiting Sequence Number Leakage: TCP Hijacking in NAT-Enabled Wi-Fi Networks[C]//31st Network and Distributed System Security Symposium (NDSS). 2024.
- [5] FENG X, LI Z, LI Q, et al. Off-Path TCP Exploits: PMTUD Breaks TCP Connection Isolation in IP Address Sharing Scenarios[C]//Proc. 2025 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2025. DOI: 10.1145/3719027.3744888.
- [6] FENG X, LI Q, SUN K, et al. Exploiting Cross-Layer Vulnerabilities: Off-Path Attacks on the TCP/IP Protocol Suite[J]. Communications of the ACM, 2025, 68(3): 48-59. DOI: 10.1145/3689819.
- [7] GERSHUNI E, AMIT N, GURFINKEL A, et al. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions[C]//Proc. 40th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI). 2019: 1069-1084. DOI: 10.1145/3314221.3314590.
- [8] ROMERA-PAREDES B, BAREKATAIN M, NOVIKOV A, et al. Mathematical Discoveries from Program Search with Large Language Models[J]. Nature, 2024, 625: 468-475. DOI: 10.1038/s41586-023-06924-6.

参考文献 II

- [9] NOVIKOV A, V N, EISENBERGER M, et al. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery[J]. ArXiv preprint arXiv:2506.13131, 2025.
- [10] Anthropic. Claude Code: Agentic Coding in the Terminal[Z]. <https://www.anthropic.com/claude-code>. 2025.
- [11] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[C]//Advances in Neural Information Processing Systems (NeurIPS). 2024.
- [12] CAO Y, WANG Z, QIAN Z, et al. Principled Unearthing of TCP Side Channel Vulnerabilities[C]//Proc. 2019 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2019: 211-224. DOI: 10.1145/3319535.3354250.
- [13] MAN K, WANG Z, HAO Y, et al. SCAD: Towards a Universal and Automated Network Side-Channel Vulnerability Detection[C]//33rd USENIX Security Symposium. 2024.
- [14] SHIROKOV N, DASINENI R. Open-sourcing Katran, a Scalable Network Load Balancer[Z]. Meta Engineering Blog. 2018.
- [15] GRAF T. EBPF – The Future of Networking and Security[Z]. Cilium / Isovalent Blog. 2020.
- [16] The Falco Authors. Falco: Cloud Native Runtime Security[Z]. CNCF Project. 2018.
- [17] Isovalent. Tetragon: eBPF-based Security Observability and Runtime Enforcement[Z]. Isovalent / CNCF. 2022.
- [18] XU Q, WONG M D, WAGLE T, et al. Synthesizing Safe and Efficient Kernel Extensions for Packet Processing[C]//Proc. 2021 ACM SIGCOMM Conference. 2021. DOI: 10.1145/3452296.3472929.
- [19] DENG G, LIU Y, MAYORAL-VILCHES V, et al. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing[C]//33rd USENIX Security Symposium. 2024: 847-864.

参考文献 III

- [20] GLAZUNOV S, BRAND M. Project Naptime: Evaluating Offensive Security Capabilities of Large Language Models[Z]. Google Project Zero Blog. 2024.
- [21] YANG Y, ZHENG Y, CHEN M, et al. Kgent: Kernel Extensions Large Language Model Agent[C]//Proc. ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions. 2024: 30-36. DOI: 10.1145/3672197.3673434.
- [22] Anyosphere. Cursor: The AI Code Editor[Z]. <https://www.cursor.com>. 2024.

敬请各位老师批评指正

谢 谢!